

一、thread库

C++11之前，C++语言没有对并发编程提供语言级别的支持，这使得我们在编写可移植的并发程序时，存在诸多的不便。现在C++11中增加了线程以及线程相关的类，很方便地支持了并发编程，使得编写的多线程程序的可移植性得到了很大的提高。

thread库底层是对各个系统的线程库进行封装，如Linux下的pthread库和Windows下Thread库等，所以C++11 thread库的第一个特点是可以跨平台，第二个特点是Linux和Windows下提供的线程库都是面向过程的，C++11 thread是库面向对象的，并且融合了一些C++11语言特点，如右值引用的移动语义，可变模板参数等，用起来会更好用一些。

1.1 构造函数

```
// 第一种
thread() noexcept;
// 第二种
thread( thread&& other ) noexcept;
// 第三种
template< class Function, class... Args >
explicit thread( Function&& f, Args&&... args );
// 第四种
thread( const thread& ) = delete;
```

1. 构造函数1：默认构造函数，构造一个线程对象，在这个线程中不执行任何处理动作
2. 构造函数2：移动构造函数，将 other 的线程所有权(比如函数返回线程对象、容器存储线程)转移给新的thread 对象。之后 other 不再表示执行线程。

3. 构造函数3：创建线程对象，并在该线程中执行函数f中的业务逻辑，args是要传递给函数f的参数。日常最常用的就是这个，他支持传一个可调用对象和参数即可，相比pthread_create而言，这里不再局限于只传递函数指针，其次就是参数传递更方便，pthread_create调用时，要传递多个参数需要打包成一个结构体，传结构体对象的指针过去。

1. 任务函数f的可选类型有很多，具体如下：

1. 普通函数，类成员函数，匿名函数，仿函数（这些都是可调用对象类型）
2. 可以是可调用对象包装器类型，也可以是使用绑定器绑定之后得到的类型（仿函数）

4. 构造函数4：使用=delete显示删除拷贝构造, 不允许线程对象之间的拷贝

1.2 公共成员函数

1.2.1 get_id()

应用程序启动之后默认只有一个线程，这个线程一般称之为主线程或父线程，通过线程类创建出的线程一般称之为子线程，每个被创建出的线程实例都对应一个线程ID，这个ID是唯一的，可以通过这个ID来区分和识别各个已经存在的线程实例，这个获取线程ID的函数叫做get_id()，函数原型如下：

```
std::thread::id get_id() const noexcept;
```

示例代码：

```
#include <iostream>
#include <thread>
#include <chrono>
using namespace std;

void func(int num, string str)
{
    for (int i = 0; i < 10; ++i)
    {
        cout << "子线程: i = " << i << "num: "
```

```

        << num << ", str: " << str << endl;
    }
}

void func1()
{
    for (int i = 0; i < 10; ++i)
    {
        cout << "子线程: i = " << i << endl;
    }
}

int main()
{
    cout << "主线程的线程ID: " << this_thread::get_id() << endl;
    thread t(func, 520, "i love you");
    thread t1(func1);
    cout << "线程t 的线程ID: " << t.get_id() << endl;
    cout << "线程t1的线程ID: " << t1.get_id() << endl;
}

```

1. `thread t(func, 520, "i love you");`: 创建了子线程对象t, `func()`函数会在这个子线程中运行
 1. `func()`是一个回调函数, 线程启动之后就会执行这个任务函数, 只需要实现即可
 2. `func()`的参数是通过`thread`的参数进行传递的, 520,i love you都是调用`func()`需要的实参
 3. 线程类的构造函数第三种是一个变参函数, 因此无需担心线程任务函数的参数个数问题
 4. 任务函数`func()`一般返回值指定为`void`, 因为子线程在调用这个函数的时候不会处理其返回值
2. `thread t1(func1);`: 子线程对象t1中的任务函数`func1()`, 没有参数, 因此在线程构造函数中就无需指定了
3. 通过线程对象调用`get_id()`就可以知道这个子线程的线程ID了, `t.get_id()`, `t1.get_id()`

在上面的示例程序中有一个bug，在主线程中依次创建出两个子线程，打印两个子线程的线程ID，最后主线程执行完毕就退出了（主线程就是执行main()函数的那个线程）。默认情况下，主线程销毁时会将其关联的两个子线程也一并销毁，但是这时有可能子线程中的任务还没有执行完毕，最后也就得不到我们想要的结果了。

当启动了一个线程（创建了一个thread对象）之后，在这个线程结束的时候（std::terminate()），我们如何去回收线程所使用的资源呢？thread库给我们两种选择：

1. 加入式（join()）
2. 分离式（detach()）

我们必须要在线程对象销毁之前在二者之间作出选择，否则程序运行期间就会有bug产生

1.2.2 join()

join()字面意思是连接一个线程，意味着主动地等待线程的终止（线程阻塞）。在某个线程中通过子线程对象调用join()函数，调用这个函数的线程被阻塞，但是子线程对象中的任务函数会继续执行，当任务执行完毕之后join()会清理当前子线程中的相关资源然后返回，同时，调用该函数的线程解除阻塞继续向下执行。

再次强调，我们一定要搞清楚这个函数阻塞的是哪一个线程，函数在哪个线程中被执行，那么函数就阻塞哪个线程。该函数的函数原型如下：

```
void join();
```

有了这样一个线程阻塞函数之后，就可以解决在上面测试程序中的bug了，如果要阻塞主线程的执行，只需要在主线程中通过子线程对象调用这个方法即可，当调用这个方法的子线程对象中的任务函数执行完毕之后，主线程的阻塞也就随之解除了。修改之后的示例代码如下：

```
int main()
{
    cout << "主线程的线程ID: " << this_thread::get_id() << endl;
    thread t(func, 520, "i love you");
    thread t1(func1);
    cout << "线程t 的线程ID: " << t.get_id() << endl;
    cout << "线程t1的线程ID: " << t1.get_id() << endl;
    t.join();
    t1.join();
}
```

当主线程运行到第八行`t.join()`；根据子线程对象`t`的任务函数`func()`的执行情况，主线程会做如下处理：

- 如果任务函数`func()`还没执行完毕，主线程阻塞，直到任务执行完毕，主线程解除阻塞，继续向下运行
- 如果任务函数`func()`已经执行完毕，主线程不会阻塞，继续向下运行

同样，第9行的代码亦如此。

1.2.3 detach()

`detach()`函数的作用是进行线程分离，分离主线程和创建出的子线程。在线程分离之后，主线程退出也会一并销毁创建出的所有子线程，在主线程退出之前，它可以脱离主线程继续独立的运行，任务执行完毕之后，这个子线程会自动释放自己占用的系统资源。（其实就是孩子翅膀硬了，和家里断绝关系，自己外出闯荡了，如果家里被诛九族还是会受牵连）。该函数函数原型如下：

```
void detach();
```

线程分离函数没有参数也没有返回值，只需要在线程成功之后，通过线程对象调用该函数即可，继续将上面的测试程序修改一下：

```

int main()
{
    cout << "主线程的线程ID: " << this_thread::get_id() << endl;
    thread t(func, 520, "i love you");
    thread t1(func1);
    cout << "线程t 的线程ID: " << t.get_id() << endl;
    cout << "线程t1的线程ID: " << t1.get_id() << endl;
    t.detach();
    t1.detach();
    // 让主线程休眠, 等待子线程执行完毕
    this_thread::sleep_for(chrono::seconds(5));
}

```

注意事项：线程分离函数`detach()`不会阻塞线程，子线程和主线程分离之后，在主线程中就不能再对这个子线程做任何控制了，比如：通过`join()`阻塞主线程等待子线程中的任务执行完毕，或者调用`get_id()`获取子线程的线程ID。有利就有弊，鱼和熊掌不可兼得，建议使用`join()`。

1.2.4 joinable()

`joinable()`函数用于判断主线程和子线程是否处理关联（连接）状态，一般情况下，二者之间的关系处于关联状态，该函数返回一个布尔类型：

1. 返回值为`true`：主线程和子线程之间有关联（连接）关系
2. 返回值为`false`：主线程和子线程之间没有关联（连接）关系

函数原型：

```

bool joinable() const noexcept;

```

示例代码：

```

#include <iostream>
#include <thread>
#include <chrono>
using namespace std;

```

```

void foo()
{
    this_thread::sleep_for(std::chrono::seconds(1));
}

int main()
{
    thread t;
    cout << "before starting, joinable: " << t.joinable() << endl;

    t = thread(foo);
    cout << "after starting, joinable: " << t.joinable() << endl;

    t.join();
    cout << "after joining, joinable: " << t.joinable() << endl;

    thread t1(foo);
    cout << "after starting, joinable: " << t1.joinable() << endl;
    t1.detach();
    cout << "after detaching, joinable: " << t1.joinable() << endl;
}

```

执行的结果：

before starting, joinable: 0

after starting, joinable: 1

after joining, joinable: 0

after starting, joinable: 1

after detaching, joinable: 0

基于结果可以得出以下结论：

1. 在创建的子线程对象的时候，如果没有指定任务函数，那么子线程不会启动，主线程和这个子线程也不会进行连接
2. 在创建的子线程对象的时候，如果指定了任务函数，子线程启动并执行任务，主线程和这个子线程自动连接成功
3. 子线程调用了detach()函数之后，父子线程分离，同时二者的连接断开，调用joinable()返回false

4. 在子线程调用了`join()`函数，子线程中的任务函数继续执行，直到任务处理完毕，这时`join()`会清理（回收）当前子线程的相关资源，所以这个子线程和主线程的连接也就断开了，因此，调用`join()`之后再调用`joinable()`会返回`false`。

二、线程命名空间 `this_thread`

在C++11中不仅添加了线程类，还添加了一个关于线程的命名空间`std::this_thread`，在这个命名空间中提供了四个公共的成员函数，通过这些成员函数就可以对当前线程进行相关的操作了。

2.1 `get_id()`

这个函数已经在上一部分介绍过了，在这里就不多赘述了。

2.2 `sleep_for()`

同样地线程被创建后也有这五种状态：创建态，就绪态，运行态，阻塞态(挂起态)，退出态(终止态)，关于状态之间的转换是一样的，在此不再过多的赘述。

线程和进程的执行有很多相似之处，在计算机中启动的多个线程都需要占用CPU资源，但是CPU的个数是有限的并且每个CPU在同一时间点不能同时处理多个任务。为了能够实现并发处理，多个线程都是分时复用CPU时间片，快速的交替处理各个线程中的任务。因此多个线程之间需要争抢CPU时间片，抢到了就执行，抢不到则无法执行（因为默认所有的线程优先级都相同，内核也会从中调度，不会出现某个线程永远抢不到CPU时间片的情况）。

命名空间`this_thread`中提供了一个休眠函数`sleep_for()`，调用这个函数的线程会马上从运行态变成阻塞态并在这种状态下休眠一定的时长，因为阻塞态的线程已经让出了CPU资源，代码也不会被执行，所以线程休眠过程中对CPU来说没有任何负担。这个函数是函数原型如下，参数需要指定一个休眠时长，是一个时间段：


```
template <class Rep, class Period>
void sleep_for (const chrono::duration<Rep,Period>& rel_time);
```

示例代码：

```
#include <iostream>
#include <thread>
#include <chrono>
using namespace std;

void func()
{
    for (int i = 0; i < 10; ++i)
    {
        this_thread::sleep_for(chrono::seconds(1));
        cout << "子线程: " << this_thread::get_id() << ", i = " << i << endl;
    }
}

int main()
{
    thread t(func);
    t.join();
}
```

在func()函数的for循环中使用了this_thread::sleep_for(chrono::seconds(1));之后，每循环一次程序都会阻塞1秒钟，也就是说每隔1秒才会进行一次输出。需要注意的是：程序休眠完成之后，会从阻塞态重新变成就绪态，就绪态的线程需要再次争抢CPU时间片，抢到之后才会变成运行态，这时候程序才会继续向下运行。

2.3 sleep_until()

命名空间this_thread中提供了另一个休眠函数sleep_until()，和sleep_for()不同的是它的参数类型不一样：

1. `sleep_until()`: 指定线程阻塞到某一个指定的时间点`time_point`类型，之后解除阻塞
2. `sleep_for()`: 指定线程阻塞一定的时间长度`duration` 类型，之后解除阻塞

函数原型:

```
template <class Clock, class Duration>
    void sleep_until (const chrono::time_point<Clock,Duration>& abs_time);
```

示例代码:

```
#include <iostream>
#include <thread>
#include <chrono>
using namespace std;

void func()
{
    for (int i = 0; i < 10; ++i)
    {
        // 获取当前系统时间点
        auto now = chrono::system_clock::now();
        // 时间间隔为2s
        chrono::seconds sec(2);
        // 当前时间点之后休眠两秒
        this_thread::sleep_until(now + sec);
        cout << "子线程: " << this_thread::get_id() << ", i = " << i << endl;
    }
}

int main()
{
    thread t(func);
    t.join();
}
```

`sleep_until()`和`sleep_for()`函数的功能是一样的，只不过前者是基于时间点去阻塞线程，后者是基于时间段去阻塞线程，项目开发过程中根据实际情况选择最优的解决方案即可。

2.4 yield()

yield()是主动让出当前线程的执行权，让其他线程先执行。此函数的确切行为依赖于实现，特别是取决于使用中的 OS 调度器机制和系统状态。例如，先进先出实时调度器（Linux 的 SCHED_FIFO）会挂起当前线程并将它放到准备运行的同优先级线程的队列尾，而若无其他线程在同优先级，则 yield 无效果。

函数原型：

```
void yield() noexcept;
```

示例代码：

```
#include <iostream>
#include <thread>
using namespace std;

void func()
{
    for (int i = 0; i < 1000000000000; ++i)
    {
        cout << "子线程: " << this_thread::get_id() << ", i = " << i << endl;
        this_thread::yield();
    }
}

int main()
{
    thread t(func);
    thread t1(func);
    t.join();
    t1.join();
}
```

在上面的程序中，执行func()中的for循环会占用大量的时间，在极端情况下，如果当前线程占用CPU资源不释放就会导致其他线程中的任务无法被处理，或者该线程每次都能抢到CPU时间片，导致其他线程中的任务没有机会被执行。解决方案就是每执行一次循环，让该线程主动放弃CPU资源，重新和其他线程再次抢夺CPU时间片，如果其他线程抢到了CPU时间片就可以执行相应的任务了。

结论：

1. std::this_thread::yield() 的目的是避免一个线程长时间占用CPU资源，从而导致多线程处理性能下降
2. std::this_thread::yield() 是让当前线程主动放弃了当前自己抢到的CPU资源，但是在下一轮还会继续抢

三、call_once

在某些特定情况下，某些函数只能在多线程环境下调用一次，比如：要初始化某个对象，而这个对象只能被初始化一次，就可以使用std::call_once()来保证函数在多线程环境下只能被调用一次。使用call_once()的时候，需要一个once_flag作为call_once()的传入参数，该函数的原型如下：

```
// 定义于头文件 <mutex>
template< class Callable, class... Args >
void call_once( std::once_flag& flag, Callable&& f, Args&&... args );
```

1. flag: once_flag类型的对象，要保证这个对象能够被多个线程同时访问到
2. f: 回调函数，可以传递一个有名函数地址，也可以指定一个匿名函数
3. args: 作为实参传递给回调函数

多线程操作过程中，std::call_once()内部的回调函数只会被执行一次，示例代码如下：

```
#include <iostream>
#include <thread>
#include <mutex>
```

```
using namespace std;

once_flag g_flag;
void do_once(int a, string b)
{
    cout << "name: " << b << ", age: " << a << endl;
}

void do_something(int age, string name)
{
    static int num = 1;
    call_once(g_flag, do_once, 19, "luffy");
    cout << "do_something() function num = " << num++ << endl;
}

int main()
{
    thread t1(do_something, 20, "ace");
    thread t2(do_something, 20, "sabo");
    thread t3(do_something, 19, "luffy");
    t1.join();
    t2.join();
    t3.join();

    return 0;
}
```

输出的结果：

name: luffy, age: 19

do_something() function num = 1

do_something() function num = 2

do_something() function num = 3

通过输出的结果可以看到，虽然运行的三个线程中都执行了任务函数do_something()但是call_once()中指定的回调函数只被执行了一次，我们的目的也达到了。

四、线程同步之互斥锁 mutex

进行多线程编程，如果多个线程需要对同一块内存进行操作，比如：同时读、同时写、同时读写对于后两种情况来说，如果不做任何的人为干涉就会出现各种各样的错误数据。这是因为线程在运行的时候需要先得到CPU时间片，时间片用完之后需要放弃已获得的CPU资源，就这样线程频繁地在就绪态和运行态之间切换，更复杂一点还可以在就绪态、运行态、挂起态之间切换，这样就会导致线程的执行顺序并不是有序的，而是随机的混乱的，就如同下图中的这个例子一样，理想很丰满现实却很残酷。



mutex是封装的互斥锁的类，用于保护临界区的共享数据。mutex主要提供lock和unlock两个接口函数。mutex 提供排他性非递归所有权语义：

- (1) 调用方线程从它成功调用 lock或 try_lock 开始，到它调用 unlock 为止占有 mutex。
- (2) 线程占有 mutex 时，其他线程如果试图要求 mutex 的所有权，那么就会阻塞（对于 lock 的调用），对于 try_lock就会返回false。

解决多线程数据混乱的方案就是进行线程同步，最常用的就是互斥锁，在C++11中共提供了四种互斥锁：

1. std::mutex ： 独占的互斥锁，不能递归使用
2. std::timed_mutex ： 带超时的独占互斥锁，不能递归使用
3. std::recursive_mutex ： 递归互斥锁，不带超时功能
4. std::recursive_timed_mutex ： 带超时的递归互斥锁

4.1 std::lock_guard

lock_guard是C++11提供的支持RAII方式管理互斥锁资源的类，这样可以更有效的防止因为异常等

原因导致的死锁问题。这个模板类的定义和常用的构造函数原型如下：

```
// 类的定义，定义于头文件 <mutex>
template< class Mutex >
class lock_guard;
// 常用构造函数
explicit lock_guard( mutex_type& m );
```

lock_guard在使用上面提供的这个构造函数构造对象时，会自动锁定互斥量，而在退出作用域后进行析构时就会自动解锁，从而保证了互斥量的正确操作，避免忘记unlock()操作而导致线程死锁。lock_guard使用了RAII技术，就是在类构造函数中分配资源，在析构函数中释放资源，保证资源出了作用域就释放。

使用lock_guard对上面的例子进行修改，代码如下：

```
void slow_increment(int id)
{
    for (int i = 0; i < 3; ++i)
    {
        // 使用哨兵锁管理互斥锁
        lock_guard<mutex> lock(g_num_mutex);
        ++g_num;
        cout << id << " => " << g_num << endl;
        this_thread::sleep_for(chrono::seconds(1));
    }
}
```

通过修改发现代码被精简了，而且不用担心因为忘记解锁而造成程序的死锁，但是这种方式也有弊端，在上面的示例程序中整个for循环的体都被当做了临界区，多个线程是线性的执行临界区代码的，因此临界区越大程序效率越低，还是需要根据实际情况选择最优的解决方案。

4.2 std::unique_lock

`unique_lock`也是C++11提供的支持RAII方式管理互斥锁资源的类，相比`lock_guard`他的功能支持

更丰富复杂。这是`unique_lock`的文档：https://legacy.cplusplus.com/reference/mutex/unique_lock/

1. `unique_lock`首先在构造的时候传时间段和时间点，用来管理`time_mutex`系统，构造时调`try_lock_for`和`try_lock_until`
2. `unique_lock`不支持拷贝和赋值，支持移动构造和移动赋值。
3. `unique_lock`还提供了`lock/try_lock/unlock`等系列的接口等系统的接口
4. `unique_lock`还可以通过`operator bool`去检查是否lock了锁对象

4.3 std::mutex

4.3.1 成员函数

`lock()`函数用于给临界区加锁，并且只能有一个线程获得锁的所有权，它有阻塞线程的作用，函数原型如下：

```
void lock();
```

独占互斥锁对象有两种状态：锁定和未锁定。如果互斥锁是打开的，调用`lock()`函数的线程会得到互斥锁的所有权，并将其上锁，其它线程再调用该函数的时候由于得不到互斥锁的所有权，就会被`lock()`函数阻塞。当拥有互斥锁所有权的线程将互斥锁解锁，此时被`lock()`阻塞的线程解除阻塞，抢到互斥锁所有权的线程加锁并继续运行，没抢到互斥锁所有权的线程继续阻塞。

除了使用`lock()`还可以使用`try_lock()`获取互斥锁的所有权并对互斥锁加锁，函数原型如下：

```
bool try_lock();
```


二者的区别在于try_lock()不会阻塞线程，lock()会阻塞线程：

1. 如果互斥锁是未锁定状态，得到了互斥锁所有权并加锁成功，函数返回true
2. 如果互斥锁是锁定状态，无法得到互斥锁所有权加锁失败，函数返回false

当互斥锁被锁定之后可以通过unlock()进行解锁，但是需要注意的是只有拥有互斥锁所有权的线程也就是对互斥锁上锁的线程才能将其解锁，其它线程是没有权限做这件事情的。该函数的函数原型如下：

```
void unlock();
```

通过介绍以上三个函数，使用互斥锁进行线程同步的大致思路差不多就能搞清楚了，主要分为以下几步：

1. 找到多个线程操作的共享资源（全局变量、堆内存、类成员变量等），也可以称之为临界资源
2. 找到和共享资源有关的上下文代码，也就是临界区
3. 在临界区的上边调用互斥锁类的 lock() 方法
4. 在临界区的下边调用互斥锁的 unlock() 方法

线程同步的目的是让多线程按照顺序依次执行临界区代码，这样做线程对共享资源的访问就从并行访问变为了线性访问，访问效率降低了，但是保证了数据的正确性。

当线程对互斥锁对象加锁，并且执行完临界区代码之后，一定要使用这个线程对互斥锁解锁，否则最终会造成线程的死锁。死锁之后当前应用程序中的所有线程都会被阻塞，并且阻塞无法解除，应用程序也无法继续运行。

4.3.2 线程同步

举个栗子，我们让两个线程共同操作同一个全局变量，二者交替数数，将数值存储到这个全局变量里边并打印出来。

```
#include <iostream>
#include <chrono>
#include <thread>
#include <mutex>
using namespace std;
```

```

int g_num = 0; // 为 g_num_mutex 所保护
mutex g_num_mutex;

void slow_increment(int id)
{
    for (int i = 0; i < 3; ++i)
    {
        g_num_mutex.lock();
        ++g_num;
        cout << id << " => " << g_num << endl;
        g_num_mutex.unlock();

        this_thread::sleep_for(chrono::seconds(1));
    }
}

int main()
{
    thread t1(slow_increment, 0);
    thread t2(slow_increment, 1);
    t1.join();
    t2.join();
}

```

在上面的示例程序中，两个子线程执行的任务是一样的（其实也可以不一样，不同的任务中也可以对共享资源进行读写操作），在任务函数中把与全局变量相关的代码加了锁，两个线程只能顺序访问这部分代码（如果不进行线程同步打印出的数据是混乱且无序的）。另外需要强调一点：

1. 在所有线程的任务函数执行完毕之前，互斥锁对象是不能被析构的，一定要在程序中保证这个对象的可用性
2. 互斥锁的个数和共享资源的个数相等，也就是说每一个共享资源都应该对应一个互斥锁对象。互斥锁对象的个数和线程的个数没有关系。

4.4 std::recursive_mutex

递归互斥锁std::recursive_mutex允许同一线程多次获得互斥锁，可以用来解决同一线程需要多次获取互斥量时死锁的问题，在下面的例子中使用独占非递归互斥量会发生死锁：

```
#include <iostream>
#include <thread>
#include <mutex>
using namespace std;

struct Calculate
{
    Calculate() : m_i(6) {}

    void mul(int x)
    {
        lock_guard<mutex> locker(m_mutex);
        m_i *= x;
    }

    void div(int x)
    {
        lock_guard<mutex> locker(m_mutex);
        m_i /= x;
    }

    void both(int x, int y)
    {
        lock_guard<mutex> locker(m_mutex);
        mul(x);
        div(y);
    }

    int m_i;
    mutex m_mutex;
};

int main()
{
    Calculate cal;
    cal.both(6, 3);
    return 0;
}
```

```
}
```

上面的程序中执行了`cal.both(6, 3);`调用之后，程序就会发生死锁，在`both()`中已经对互斥锁加锁了，继续调用`mul()`函数，已经得到互斥锁所有权的线程再次获取这个互斥锁的所有权就会造成死锁（在C++中程序会异常退出，使用C库函数会导致这个互斥锁永远无法被解锁，最终阻塞所有的线程）。要解决这个死锁的问题，一个简单的办法就是使用递归互斥锁`std::recursive_mutex`，它允许一个线程多次获得互斥锁的所有权。修改之后的代码如下：

```
#include <iostream>
#include <thread>
#include <mutex>
using namespace std;

struct Calculate
{
    Calculate() : m_i(6) {}

    void mul(int x)
    {
        lock_guard<recursive_mutex> locker(m_mutex);
        m_i *= x;
    }

    void div(int x)
    {
        lock_guard<recursive_mutex> locker(m_mutex);
        m_i /= x;
    }

    void both(int x, int y)
    {
        lock_guard<recursive_mutex> locker(m_mutex);
        mul(x);
        div(y);
    }

    int m_i;
    recursive_mutex m_mutex;
};
```

```
int main()
{
    Calculate cal;
    cal.both(6, 3);
    cout << "cal.m_i = " << cal.m_i << endl;
    return 0;
}
```

虽然递归互斥锁可以解决同一个互斥锁频繁获取互斥锁资源的问题，但是还是建议少用，主要原因如下：

1. 使用递归互斥锁的场景往往都是可以简化的，使用递归互斥锁很容易放纵复杂逻辑的产生，从而导致bug的产生
2. 递归互斥锁比非递归互斥锁效率要低一些
3. 递归互斥锁虽然允许同一个线程多次获得同一个互斥锁的所有权，但最大次数并未具体说明，一旦超过一定的次数，就会抛出std::system错误。

4.5 std::timed_mutex

std::timed_mutex是超时独占互斥锁，主要是在获取互斥锁资源时增加了超时等待功能，因为不知道获取锁资源需要等待多长时间，为了保证不一直等待下去，设置了一个超时时长，超时后线程就可以解除阻塞去做其他事情了。

std::time_mutex跟mutex完全类似，只是额外提供try_lock_for和try_lock_until的接口，这两个接口跟try_lock类似，只是他不会马上返回，而是直接进入阻塞，直到时间条件到了或者解锁了就会唤醒试图获取锁资源。

```

void lock();
bool try_lock();
void unlock();

// std::timed_mutex比std::_mutex多出的两个成员函数
template <class Rep, class Period>
    bool try_lock_for (const chrono::duration<Rep,Period>& rel_time);

template <class Clock, class Duration>
    bool try_lock_until (const chrono::time_point<Clock,Duration>& abs_time);

```

1. try_lock_for 函数是当线程获取不到互斥锁资源的时候，让线程阻塞一定的时间长度
2. try_lock_until 函数是当线程获取不到互斥锁资源的时候，让线程阻塞到某一个指定的时间点
3. 关于两个函数的返回值：当得到互斥锁的所有权之后，函数会马上解除阻塞，返回true，如果阻塞的时长用完或者到达指定的时间点之后，函数也会解除阻塞，返回false

std::timed_mutex示例代码:

```

#include <iostream>
#include <thread>
#include <mutex>
using namespace std;

timed_mutex g_mutex;

void work()
{
    chrono::seconds timeout(1);
    while (true)
    {
        // 通过阻塞一定的时长来争取得到互斥锁所有权
        if (g_mutex.try_lock_for(timeout))
        {
            cout << "当前线程ID: " << this_thread::get_id()
                 << ", 得到互斥锁所有权..." << endl;
            // 模拟处理任务用了一定的时长
            this_thread::sleep_for(chrono::seconds(10));
        }
    }
}

```


在上面的例子中，通过一个while循环不停的去获取超时互斥锁的所有权，如果得不到就阻塞1秒钟，1秒之后如果还是得不到阻塞50毫秒，然后再次继续尝试，直到获得互斥锁的所有权，跳出循环体。

4.6 std::recursive_timed_mutex

递归超时互斥锁std::recursive_timed_mutex的使用方式和std::timed_mutex是一样的，只不过它可以允许一个线程多次获得互斥锁所有权，而std::timed_mutex只允许线程获取一次互斥锁所有权。另外，递归超时互斥锁std::recursive_timed_mutex也拥有和std::recursive_mutex一样的弊端，不建议频繁使用。

五、线程同步之条件变量 condition_variable

条件变量是C++11提供的另外一种用于等待的同步机制，它能阻塞一个或多个线程，直到收到另外一个线程发出的通知或者超时，才会唤醒当前阻塞的线程。

condition_variable需要配合互斥锁系列进行使用，主要提供wait和notify系统接口。，C++11提供了两种条件变量：

1. condition_variable: 需要配合std::unique_lock [std::mutex](#) 进行wait操作，也就是阻塞线程的操作。
2. condition_variable_any: condition_variable_any 类是 std::condition_variable 的泛化。相对于只在std::unique_lock [std::mutex](#) 上工作的 std::condition_variable，condition_variable_any 可以和任意带有lock()、unlock()语义的mutex搭配使用。

条件变量通常用于生产者和消费者模型，大致使用过程如下：

1. 拥有条件变量的线程获取互斥量
2. 循环检查某个条件，如果条件不满足阻塞当前线程，否则线程继续向下执行
 - 产品的数量达到上限，生产者阻塞，否则生产者一直生产。。。
 - 产品的数量为零，消费者阻塞，否则消费者一直消费。。。

3. 条件满足之后，可以调用`notify_one()`或者`notify_all()`唤醒一个或者所有被阻塞的线程

- 由消费者唤醒被阻塞的生产者，生产者解除阻塞继续生产。。。
- 由生产者唤醒被阻塞的消费者，消费者解除阻塞继续消费。。。

5.1 condition_variable

5.1.1 成员函数

`condition_variable`的成员函数主要分为两部分：线程等待（阻塞）函数 和线程通知（唤醒）函数，这些函数被定义于头文件 `<condition_variable>`。

1. 等待函数

调用`wait()`函数的线程会被阻塞

```
// 1
void wait (unique_lock<mutex> &lck);
// 2
template <class Predicate>
void wait (unique_lock<mutex> &lck, Predicate pred);
```

- 函数1：调用该函数的线程直接被阻塞
- 函数2：该函数的第二个参数是一个判断条件，是一个返回值为布尔类型的函数
 - 该参数可以传递一个有名函数的地址，也可以直接指定一个匿名函数
 - 表达式返回`false`当前线程被阻塞，表达式返回`true`当前线程不会被阻塞，继续向下执行
- 如果线程被该函数阻塞，这个线程会释放占有的互斥锁的所有权，当阻塞解除之后这个线程会重新得到互斥锁的所有权，继续向下执行

2. 通知函数

```
void notify_one() noexcept;
void notify_all() noexcept;
```

- `notify_one()`: 唤醒一个被当前条件变量阻塞的线程
- `notify_all()`: 唤醒全部被当前条件变量阻塞的线程

5.2 condition_variable_any

5.2.1 成员函数

`condition_variable_any`的成员函数也是分为两部分：线程等待（阻塞）函数 和线程通知（唤醒）函数，这些函数被定义于头文件 `<condition_variable>`。

1. 等待函数

```
// 1
template <class Lock> void wait (Lock& lck);
// 2
template <class Lock, class Predicate>
void wait (Lock& lck, Predicate pred);
```

函数参数和条件和`condition_variable`一样。

2. 通知函数

函数参数和条件和`condition_variable`一样。

3. 总结：

以上介绍的两种条件变量各自有各自的特点，`condition_variable` 配合`unique_lock` 使用更灵活一些，可以在任何时候自由地释放互斥锁，而`condition_variable_any` 如果和`lock_guard` 一起使用必须要等到其生命周期结束才能将互斥锁释放。但是，`condition_variable_any` 可以和多种互斥锁配合使用，应用场景也更广，而`condition_variable` 只能和独占的非递归互斥锁（`mutex`）配合使用，有一定的局限性。

六、线程同步之原子变量atomic

6.1 为什么需要原子变量

在多线程编程中，对普通共享变量的“看似简单”的操作（比如`count++`），实际上是**非原子的**——它会被拆分成 3 个 CPU 指令：

1. 从内存读取`count` 的值到 CPU 寄存器；
2. 寄存器中值加 1
3. 把寄存器的值写回内存

如果多个线程同时执行`count++`，就会出现竞态条件（**Race Condition**）：比如线程 A 和 B 同时读取到`count=0`，各自加 1 后写回，最终`count=1`而非预期的2。

传统解决方式是用 `std::mutex` 加锁，但锁的开销较大（涉及内核态切换）；而 `std::atomic` 通过**硬件级别的原子指令**实现操作的“不可中断性”，无需手动加锁，效率更高。

6.2 atomic

原子操作： 一个操作从开始到结束，中间不会被任何其他线程中断，要么完全执行完毕，要么完全不执行，不存在“执行一半”的状态

`atomic`的原理主要是硬件层面的支持，现代处理器提供了原子指令来支持原子操作。例如，在 x86架构中有CMPXCHG（比较并交换）指令。这些原子指令能够在一个不可分割的操作中完成对内存的读取、比较和写入操作，简称CAS，Compare And Set，或是 Compare And Swap。另外为了处理多个处理器缓存之间的数据一致性问题，硬件采用了缓存一致性协议，当一个`atomic`操作修改了一个变量的值，缓存一致性协议会确保其他处理器缓存中的相同变量副本被正确地更新或标记为无效。

6.2.1 构造函数

```
// 1
atomic() noexcept = default;
// 2
constexpr atomic( T desired ) noexcept;
// 3
atomic( const atomic& ) = delete;
```

1. 构造函数1：默认无参构造函数。
2. 构造函数2：使用 desired 初始化原子变量的值。
3. 构造函数3：使用=delete显示删除拷贝构造函数, 不允许进行对象之间的拷贝

6.2.2 成员函数

1. 读操作: load() -- 原子读取变量值
2. 写操作: store(val) -- 原子写入值val到变量
3. 加减操作:fetch_add(n)/fetch_sub(n)--- 原子加 n/ 减n, 返回操作前的值
4. ...其他的成员函数可以查看[官方文档](#)

6.2.3 示例

多线程累加问题 示例代码:

```
// 原子变量, 操作是原子的
atomic<int> atomic_count(0);
void increment_atomic() {
    for (int i = 0; i < 100000; ++i) {
        // 方式1: 运算符重载
        atomic_count++;
        // 方式2: 成员函数
        // atomic_count.fetch_add(1);
    }
}

int main() {
    thread t1(increment_atomic);
    thread t2(increment_atomic);
```

```
t1.join();
t2.join();
// 结果必然是200000，无竞态条件
cout << "原子变量累加结果：" << atomic_count << endl;
return 0;
}
```

七、多线程异步

举一个例子来解释以下异步：

我们以“周末做家务”这个场景，对比同步和异步的区别：

1. 同步调用方式

你一个人做家务，任务清单是：

- 把脏衣服放进洗衣机，启动洗涤程序（耗时 30 分钟）
- 擦桌子、拖地（耗时 15 分钟）
- 给绿植浇水（耗时 5 分钟）

同步模式下，你会：

- 先启动洗衣机，然后站在洗衣机旁一直等 30 分钟，直到洗衣程序结束，才开始擦桌子、拖地；
- 等擦完地（15 分钟），再去给绿植浇水（5 分钟）。
- 总耗时：30+15+5 = 50 分钟。
- 核心特点：做完一件事，再做下一件，中间等待的时间完全浪费，效率极低。

2. 异步调用方式

还是同样的任务清单，异步模式下，你会：

- 先启动洗衣机（相当于“创建一个子线程处理耗时任务”），不用等它完成，转身立刻去擦桌子、拖地；
- 擦完地后，接着给绿植浇水
- 等你做完擦地、浇水（总共 20 分钟），洗衣机还在运行，你可以歇 10 分钟，等洗衣机完成（30 分钟到点），再去晾衣服即可

- 核心特点：耗时的任务交给“子线程”（洗衣机）独立处理，主线程（你）继续做其他事，不用空等，最大化利用时间，效率大幅提升

7.1 std::future(未来对象)

C++11中增加的线程类，使得我们能够非常方便的创建和使用线程，但有时会有些不方便，比如需要获取线程返回的结果，就不能通过`join()`得到结果，只能通过一些额外手段获得，比如：定义一个全局变量，在子线程中赋值，在主线程中读这个变量的值，整个过程比较繁琐。C++提供的线程库中提供了一些类用于访问异步操作的结果

`std::future`是一个“结果容器”——它代表一个尚未完成的异步操作的结果。它本身不能直接创建有效的异步任务关联对象（需配合

`std::async`/`std::packaged_task`/`std::promise`使用）你可以把它理解成：“我现在拿不到结果，但我先占个位置，等异步任务做完了，就能从这个容器里取出结果”。

核心特性：

1. 一个`future`对象只能被一个线程获取一次结果（移动语义，不能拷贝）；
2. 可以通过`future`等待异步任务完成、获取任务结果、检查任务状态。

7.1.1 构造函数

```
// ①
future() noexcept;

// ②
future( future&& other ) noexcept;

// ③
future( const future& other ) = delete;
```

1. 构造函数①：默认无参构造函数,创建空的 `future` 对象，不关联任何异步任务
2. 构造函数②：移动构造函数,转移另一个 `future` 的所有权，原对象变为无效
3. 构造函数③：使用`=delete`显示删除拷贝构造函数,不允许进行对象之间的拷贝

7.1.2 常用成员函数

1. `get()` : 获取异步任务的结果（如果任务没完成，会阻塞当前线程直到结果返回）；只能调用一次，调用后`future`变为无效状态
2. `wait()` : 等待任务完成，但不获取结果（阻塞当前线程）
3. `wait_for(时长)` : 等待指定时长，返回`future_status`
4. `wait_until(时间点)` : 等待到指定时间点，返回`future_status`
5. `valid()` : 判断`future`是否有效（比如`get()`调用后会失效）

7.1.3 `std::future_status`(未来对象状态)

`future_status`是一个枚举类型，用于表示`future`对应的异步任务的状态，通常和`wait_for()/wait_until()`配合使用，实现“非阻塞等待”或“限时等待”

枚举值说明：

1. `future_status::ready`: 任务已完成，结果可用
2. `future_status::timeout`: 等待超时，任务仍未完成
3. `future_status::deferred`: 任务是延迟执行（`launch::deferred`），尚未开始

7.2 `std::async`(异步启动函数)

`std::async` 是 C++11 引入的异步执行函数，核心功能是：在后台（新线程 / 当前线程延迟）执行一个可调用对象（函数、`lambda`、仿函数等），并返回`std::future` 对象，通过该对象可以获取异步任务的结果(通过`future`类的`get()`)或异常。

该函数的原型如下：

```
// 定义于头文件 <future>
// ①
template< class Function, class... Args>
std::future<std::result_of_t<std::decay_t<Function>(std::decay_t<Args>...)>>>
    async( Function&& f, Args&&... args );

// ②
template< class Function, class... Args >
std::future<std::result_of_t<std::decay_t<Function>(std::decay_t<Args>...)>>>
    async( std::launch policy, Function&& f, Args&&... args );
```

可以看到这是一个模板函数，在C++11中这个函数有两种调用方式：

1. 函数①：直接调用传递到函数体内部的可调用对象，返回一个future对象
2. 函数②：通过指定的策略调用传递到函数内部的可调用对象，返回一个future对象

函数参数：

1. f:可调用对象，这个对象在子线程中被作为任务函数使用
2. Args:传递给 f 的参数（实参）
3. policy:可调用对象·f的执行策略

执行策略（std::launch）

- std::launch::async：强制创建新线程，立即异步执行任务；任务的执行与 future 的访问解耦。
- std::launch::deferred：延迟执行任务；直到调用 future.get()/future.wait() 时，任务才在当前线程执行（无新线程）。

7.2.1 std::launch::async策略

调用async()函数直接创建线程执行任务：

```
#include <iostream>
#include <future>
#include <chrono>

int calc_sum(int a, int b) {
    std::cout << "任务执行线程ID: " << std::this_thread::get_id() << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(2));
    return a + b;
}

int main() {
    std::cout << "主线程ID: " << std::this_thread::get_id() << std::endl;

    // 1. 异步执行任务（强制创建新线程）
    std::future<int> fut = std::async(std::launch::async, calc_sum, 3, 7);

    // 2. 主线程可并行执行其他操作
```



```

std::cout << "主线程继续执行其他任务..." << std::endl;
std::this_thread::sleep_for(std::chrono::seconds(1));

// 3. 获取异步任务结果（阻塞等待任务完成，若未完成则等待）
int result = fut.get();
std::cout << "异步任务结果：" << result << std::endl;

return 0;
}

```

执行结果：

```

主线程ID：1407092608
主线程继续执行其他任务...
任务执行线程ID：1407090560
异步任务结果：10

```

7.2.2 std::launch::deferred

调用async()函数不创建线程执行任务

```

int main() {
    std::cout << "主线程ID：" << std::this_thread::get_id() << std::endl;

    // 延迟执行策略：任务不会立即执行
    std::future<int> fut = std::async(std::launch::deferred, calc_sum, 5, 8);

    std::cout << "主线程执行中，任务尚未启动..." << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));

    // 调用 get() 时，任务才在主线程执行
    int result = fut.get();
    std::cout << "异步任务结果：" << result << std::endl; // 输出 13
    std::cout << "任务执行线程ID（与主线程一致）：" << std::this_thread::get_id()
    << std::endl;

    return 0;
}

```

执行结果：

```
主线程ID: 1407092608
主线程执行中，任务尚未启动...
任务执行线程ID: 1407092608
异步任务结果: 13
任务执行线程ID（与主线程一致）：1407092608
```

`async`是 C++11 提供的最简单的异步任务启动工具——它帮你封装了“创建线程 + 管理任务 + 返回 `future`”的全部逻辑，无需手动创建`thread`对象，是新手最容易上手的异步方式。

7.3 `std::promise`(承诺对象)

`std::promise`是一个协助线程赋值的类,生产者线程通过 `promise` 设置结果，消费者线程通过 `promise` 关联的 `future` 获取结果。可以理解成：`promise`是“写结果的容器”，`future`是“读结果的容器”，两者一一绑定，线程安全。

适合场景：

当你需要手动控制异步任务的结果返回（比如自己创建线程，而非用 `async`）时，`promise`是最佳选择。

7.3.1 构造函数

```
// ①
promise();

// ②
promise( promise&& other ) noexcept;

// ③
promise( const promise& other ) = delete;
```

1. 构造函数①：默认构造函数，创建一个空的 `promise` 对象，关联一个“未就绪”的异步状态（无结果、无异常）；

2. 构造函数②：移动构造函数,转移另一个 promise 对象的 异步状态所有权； 移动后，原对象 other 变为“空状态”（valid() 为 false），无法再设置结果 / 获取 future;
3. 构造函数③：使用=delete显示删除拷贝构造函数, 不允许进行对象之间的拷贝

7.3.2 常用成员函数

1. get_future():获取和当前 promise 绑定的 future 对象（一一对应，只能调用一次）
2. set_value(val): 设置结果值，此时关联的 future 变为 ready 状态（只能调用一次）
3. set_exception(e): 设置异常（如果任务执行出错，可通过这个传递异常，future.get() 会抛出该异常）

7.3.3 promise的使用

通过promise传递数据的过程一共分为5步：

1. 在主线程中创建std::promise对象
2. 将这个std::promise对象通过引用的方式传递给子线程的任务函数
3. 在子线程任务函数中给std::promise对象赋值
4. 在主线程中通过std::promise对象取出绑定的future实例对象
5. 通过得到的future对象取出子线程任务函数中返回的值。

示例代码：

```
#include <iostream>
#include <future>
#include <thread>
#include <chrono>
using namespace std;

// 生产者线程函数：接收promise的引用
void producer(promise<int>& prom) {
    cout << "生产者线程开始计算..." << endl;
    this_thread::sleep_for(chrono::seconds(2));
    int result = 100 * 20;
    prom.set_value(result);
    cout << "生产者线程已设置结果" << endl;
}
```

```

int main() {
    promise<int> prom;
    future<int> fut = prom.get_future();

    // 用ref包装prom, 传递引用
    thread t(producer, ref(prom));

    cout << "主线程等待结果..." << endl;
    int res = fut.get();
    cout << "主线程获取到结果: " << res << endl;

    t.join();
    return 0;
}

```

输出结果:

```

主线程等待结果...
生产者线程开始计算...
(等待2秒后)
生产者线程已设置结果
主线程获取到结果: 2000

```

7.4 std::packaged_task(任务包装器)

`packaged_task`是“可调用对象 + `future`”的封装：它把一个函数（或可调用对象，比如 `lambda`、函数指针）包装起来，并且能返回一个 `future`，用于获取该函数的执行结果。

可以理解成：`packaged_task`是“任务的容器”，把任务和 `future` 绑定，支持手动控制任务的执行时机（比如在指定线程执行）。

应用场景：

比 `async` 更灵活——`async` 是“启动任务 + 返回 `future`”，而 `packaged_task` 只“包装任务”，执行时机由你决定（比如放到线程池、延迟执行）。

7.4.1 构造函数

```
// ①
packaged_task() noexcept;

// ②
template <class F>
explicit packaged_task( F&& f );

// ③
packaged_task( const packaged_task& ) = delete;

// ④
packaged_task( packaged_task&& rhs ) noexcept;
```

1. 构造函数①：无参构造，构造一个无任务的空对象
2. 构造函数②：通过一个可调用对象，构造一个任务对象
3. 构造函数③：显示删除，不允许通过拷贝构造函数进行对象的拷贝
4. 构造函数④：移动构造函数

7.4.2 常用成员函数

1. `get_future()`: 获取绑定的 `future` 对象（只能调用一次）
2. `valid()`: 判断是否包装了有效的任务
3. `reset()`: 重置任务状态，可重新执行（需重新获取 `future`）

7.4.3 `packaged_task`的使用

`packaged_task`其实就是对子线程要执行的任务函数进行了包装，和可调用对象包装器的使用方法相同，包装完毕之后直接将包装得到的任务对象传递给线程对象就可以了。

示例代码：

```
int main() {
    // 1. 包装一个lambda表达式（计算两数之和）
    // <>里的第一个int :packaged_task包装的任务函数的返回值类型；
    // 第二、三个int：任务函数的第一、二个参数的类型；
    packaged_task<int(int, int)> task([](int a, int b) {
        cout << "task执行中..." << endl;
```

```

    this_thread::sleep_for(chrono::seconds(2));
    return a + b;
});

// 2. 获取绑定的future
future<int> fut = task.get_future();

// 3. 执行任务：方式1——直接在主线程执行
// task(10, 20);

// 方式2——在新线程执行（更常用）
thread t(move(task), 10, 20); // 注意：packaged_task不可拷贝，需移动

cout << "主线程等待task执行结果..." << endl;
int res = fut.get();
cout << "task执行结果：" << res << endl; // 输出30

t.join();
return 0;
}

```

7.5 总结

`std::async`、`std::promise`、`std::packaged_task` 三者都是 C++11 异步编程的工具，最终都通过 `std::future` 获取异步任务的结果，但设计目标和使用场景完全不同：

1. `async`：「快捷方式」—— 一键异步执行任务，无需手动管理线程 / 结果；
2. `promise`：「手动结果控制器」—— 手动设置异步结果 / 异常，灵活控制结果传递；
3. `packaged_task`：「任务包装器」—— 把可调用对象包装成“能返回 `future` 的任务”，可手动调度执行。

详细对比：

特性	** std::async **	** std::promise **	** std::packaged_task **
核心作用	异步执行任务，自动关联 future	手动设置结果 / 异常，关联 future	包装可调用对象，关联 future，手动控制执行时机
创建 future 方式	直接返回 future	调用 get_future() 获取 future	调用 get_future() 获取 future
任务执行时机	1. 显式 async 策略：立即创建新线程执行； 2. 显式 deferred 策略，调用 get()/wait() 时执行；	无内置任务，需手动在线程中调用 set_value()/set_exception() 设置结果	包装的任务不会自动执行，需手动调用（如 task()）或传递给线程执行
是否创建线程	可选（async 策略创建，deferred 不创建）	不创建线程，需手动结合 std::thread 使用	不创建线程，需手动结合 std::thread 使用
优点	1. 最简洁：一行代码实现异步执行 + 结果获取； 2. 无需手动管理线程 / 结果传递； 3. 自动捕获任务异常	1. 极致灵活：可在任意时机、任意线程设置结果 / 异常； 2. 适配复杂异步逻辑（如多阶段任务、条件触发结果	1. 任务与执行解耦：先包装任务，再决定何时 / 何地执行； 2. 可重复使用（包装后可多次调用，每次返回新 future）； 3. 适配线程池场景

特性	** std::async **	** std::promise **	** std::packaged_task **
缺点	1. 灵活性差：无法手动控制结果设置时机； 2. 默认策略下 future 临时对象析构会阻塞； 3. 无法复用任务	1. 需手动管理线程和结果设置，代码量多； 2. 需注意引用 / 移动的生命周期问题； 3. 仅能设置一次结果（多次调用 set_value 会抛异常）	1. 比 async 繁琐，需手动包装 + 执行； 2. 包装后任务状态不可逆（执行后无法重置）； 3. 需注意移动语义（不可拷贝）
适用场景	1. 简单异步任务（如单次耗时计算、IO 操作）； 2. 快速实现“后台执行 + 获取结果”，无需复杂控制； 3. 不需要复用任务的场景	1. 复杂异步逻辑（如任务分阶段执行，满足条件才设置结果）； 2. 线程间手动传递结果 / 异常； 3. 需精准控制结果设置时机的场景	1. 需复用异步任务（如多次执行相同逻辑）； 2. 手动调度任务（如放入线程池）； 3. 任务执行时机需灵活控制（如先包装，延迟执行）